第9章：Fork智能体与提示缓存

95%的洞察

当父智能体并行生成五个子智能体时，每个子智能体的API请求中绝大多数是相同的。系统提示相同。工具定义相同。对话历史相同。触发派生的助手消息相同。唯一不同的是最终指令："你处理数据库迁移"、"你编写测试"、"你更新文档"。

在典型的fork中，如果对话是热的，共享前缀可能是80,000个token。每个子智能体的指令可能是200个token。这是99.75%的重叠。Anthropic的提示缓存对缓存输入token提供90%的折扣。如果你能让这些80,000个token对第2到第5个子智能体命中缓存，你就将这四个请求的输入成本削减了90%。对于父智能体来说，这是花费4美元与花费0.50美元在相同并行分派上的区别。

问题是提示缓存是字节精确的。不是"足够相似"。不是"语义等效"。字节必须逐字匹配，从系统提示的第一个字节到每个子智能体内容分歧前的最后一个字节。一个额外的空格、一个重新排序的工具定义、一个更改系统提示片段的陈旧功能标志——缓存就会未命中。整个前缀以全价重新处理。

Fork智能体是Claude Code对这一约束的回答。它们不仅仅是"用上下文生成子智能体"的便利——它们是伪装成编排功能的提示缓存利用机制。fork系统中的每个设计决策都追溯到一个问题：我们如何保证并行子智能体之间的字节相同前缀？

Fork子智能体继承什么

Fork智能体从父智能体继承四样东西，它通过引用或字节精确副本而非重新计算来继承它们。

1. 系统提示。 不是重新生成——而是线程传递。父智能体已渲染的系统提示字节通过`override.systemPrompt`传递，从`toolUseContext.renderedSystemPrompt`拉取。这是父智能体最近API调用中发送的确切字符串。

2. 工具定义。 Fork智能体定义声明`tools: ['*']`，但将`useExactTools`标志设置为true，子智能体直接接收父智能体的组装工具数组。无过滤、无重新排序、无重新序列化。

3. 对话历史。 父智能体与API交换的每条消息——用户回合、助手回合、工具调用、工具结果——都通过`forkContextMessages`克隆到子智能体的上下文中。

4. 思考配置和模型。 Fork定义指定`model: 'inherit'`，解析为父智能体的确切模型。相同模型意味着相同的分词器、相同的上下文窗口、相同的缓存命名空间。

Fork智能体定义本身是最小的——几乎是空操作：

Fork智能体定义故意最小——它从父智能体继承一切。它指定所有工具（`'*'`），继承父智能体的模型，对权限使用bubble模式（因此提示在父智能体的终端中显示），并提供一个无操作的系统提示函数，实际上从不被调用——真正的提示通过覆盖通道到达，已经渲染且字节稳定。

字节相同前缀技巧

对Claude的API请求有特定结构：系统提示，然后工具，然后消息。为了让提示缓存命中，从请求开始到某个前缀边界的每个字节必须在请求之间相同。

Fork智能体通过确保三层被冻结来实现这一点：

第1层：通过线程传递而非重新计算的系统提示。

当父智能体的系统提示为其上次API调用渲染时，结果被捕获在`toolUseContext.renderedSystemPrompt`中。这是经过所有动态插值后的字符串——GrowthBook功能标志、环境详情、MCP服务器描述、技能内容、CLAUDE.md文件。Fork子智能体接收这个确切的字符串。

为什么不只是再次调用`getSystemPrompt()`？因为系统提示生成不是纯的。GrowthBook标志在SDK获取远程配置时从冷态过渡到热态。在父智能体第一回合返回`false`的标志可能在fork子智能体启动时返回`true`。如果系统提示包含由该标志门控的条件块，重新渲染的提示即使相差一个字符也会不同。缓存被破坏。80,000个token的全价重新处理，乘以五个子智能体。

线程传递渲染的字节消除了这一整类分歧。

第2层：通过精确传递的工具定义。

普通子智能体经历`resolveAgentTools()`，它基于智能体定义的`tools`和`disallowedTools`数组过滤工具池，应用权限模式差异，并可能重新排序工具。结果序列化的工具数组会与父智能体的不同——不同的子集、不同的顺序、不同的权限注释。

Fork智能体完全跳过这一步：

```
const resolvedTools = useExactTools
  ? availableTools  // 父智能体的确切数组
  : resolveAgentTools(agentDefinition, availableTools, isAsync).resolvedTools
```

`useExactTools`标志仅在fork路径上设置为true。子智能体按原样获得父智能体的工具池。相同的工具、相同的顺序、相同的序列化。这包括即使子智能体被禁止使用也将Agent工具保留在子智能体的池中——移除它会改变工具数组并破坏缓存。

第3层：消息数组构造。

这是`buildForkedMessages()`完成其谨慎工作的地方。该函数构造位于共享历史和每个子智能体指令之间的最后两条消息：

`buildForkedMessages()`函数构造位于共享历史和每个子智能体指令之间的最后两条消息。算法：

1. 克隆父智能体的助手消息（保留所有具有原始ID的`tool_use`块）。
2. 对于每个`tool_use`块，创建一个具有常量占位符字符串的`tool_result`（在所有子智能体中相同）。
3. 构建包含所有占位符结果的单条用户消息，后跟包装在样板标签中的每个子智能体指令。
4. 返回`[clonedAssistantMessage, userMessageWithPlaceholdersAndDirective]`。

```
// 伪代码—说明消息构造
function buildChildMessages(directive, parentAssistant) {
  const cloned = cloneMessage(parentAssistant)
  const placeholders = parentAssistant.toolUseBlocks.map(b =>
    toolResult(b.id, CONSTANT_PLACEHOLDER)  // 跨子智能体字节相同
  )
  const userMsg = createUserMessage([...placeholders, wrapDirective(directive)])
  return [cloned, userMsg]
}
```

每个子智能体的结果消息数组看起来像这样：

```
[...shared_history, assistant(all_tool_uses), user(placeholder_results...,
directive)]
```

指令前的每个元素在子智能体之间都是相同的。`FORK_PLACEHOLDER_RESULT`——常量字符串`'Fork started -- processing in background'`——确保即使工具结果块也是字节相同的。`tool_use_id`值是相同的，因为它们引用相同的助手消息。只有包含每个子智能体指令的最终文本块不同。

缓存边界正好落在最终文本块之前。其上的一切——可能是数万个token的系统提示、工具定义、对话历史和占位符结果——对第一个之后的每个子智能体以90%的折扣命中缓存。

Fork样板标签

每个子智能体的指令被包装在一个样板XML标签中，它有两个目的：它指示子智能体如何行为，并作为递归fork检测的标记。

样板包含大约10条规则。关键的有：

- **覆盖父智能体的fork指令。** 父智能体的系统提示说"默认fork"——样板明确告诉子智能体："该指令是给父智能体的。你就是fork。不要生成子智能体。"
- **静默执行，报告一次。** 工具调用之间没有对话文本。直接使用工具，然后产生结构化摘要。
- **保持在范围内。** 子智能体不得扩展到其指令之外。
- **结构化输出格式。** 响应必须遵循Scope/Result/Key files/Files changed/Issues模板，使结果易于解析当多个子智能体同时报告回来时。

规则1特别有趣。父智能体的系统提示——fork子智能体出于缓存原因逐字继承——包含诸如"有并行工作时默认fork"的指令。如果子智能体遵循该指令，它会尝试fork自己的子智能体，创建智能体的无限递归。样板明确覆盖："该指令是给父智能体的。你就是fork。"

结构化输出格式（Scope/Result/Key files/Files changed/Issues）不是装饰性的。它将子智能体的输出限制为事实报告，当五个子智能体同时报告回来时，这使结果更容易让父智能体解析和聚合。

递归Fork预防

Fork子智能体在其工具池中保留Agent工具。它必须这样做——移除它会改变序列化的工具数组并破坏提示缓存。但如果子智能体在没有`subagent_type`的情况下实际调用Agent工具，fork路径会再次触发，创建孙智能体fork。这个孙智能体会继承更大的上下文（父智能体+子智能体对话），生成自己的fork，如此往复。

两个守卫防止这一点：

主要守卫：querySource检查。 当fork子智能体被生成时，其`context.options.querySource`被设置为`'agent:builtin:fork'`。`call()`方法在允许fork路径之前检查这一点：

```
// 在AgentTool.call()中：
if (effectiveType === undefined) {
  // Fork路径—但我们已经在fork中了吗？
  if (querySource === 'agent:builtin:fork') {
    // 拒绝：已经是fork子智能体
  }
}
```

这是快速路径。它检查选项对象中的单个字符串。

备用守卫：消息扫描。 Fork预防使用两个守卫：在生成时设置的`querySource`标签（快速路径——单个字符串比较），以及扫描消息历史以查找样板XML标签的备用。备用存在是因为`querySource`在自动压缩中幸存，但在未正确线程传递的边缘情况下，消息扫描备用捕获递归。这是一种belt-and-suspenders方法，其中检查的成本（扫描消息）与意外递归fork的成本（失控的API支出）相比微不足道。

为什么有备用？因为Claude Code有autocompact功能，当上下文过长时重写消息数组。Autocompact可以重写消息内容但保留选项中的`querySource`。理论上，`querySource`单独就足够了。在实践中，消息扫描备用捕获`querySource`未正确线程传递的边缘情况——一种belt-and-suspenders方法，其中检查的成本（扫描消息）与意外递归fork的成本（失控的API支出）相比微不足道。

同步到异步转换

Fork子智能体开始在前景运行：其消息流式传输到父智能体的终端，父智能体阻塞等待完成。但如果子智能体花费太长时间怎么办？Claude Code允许执行中后台化——用户（或自动超时）可以在不丢失任何工作的情况下将运行的前景智能体推入后台。

机制出奇地干净：

1. 当前景智能体通过`registerAgentForeground()`注册时，创建后台信号promise。
2. 父智能体的同步循环在智能体的消息流和后台信号之间竞速：

```
while (true) {
  const result = await Promise.race([
    iterator.next(),          // 来自智能体的下一条消息
    backgroundSignal,         // "移至后台"触发器
  ])
  if (result === BACKGROUND_SIGNAL) break
  // ... 处理消息
}
```

3. 当后台信号触发时，前景迭代器通过 `iterator.return()` 优雅地终止。这触发生成器的 `finally` 块，处理清理。
4. 使用相同的智能体ID和累积的消息历史生成新的 `runAgent()` 实例，其中 `isAsync: true` 。智能体从离开的地方继续，现在在后台运行。
5. 原始同步 `call()` 返回 `{ status: 'async_launched' }` ，父智能体继续其对话。

没有工作丢失，因为消息历史就是智能体的状态。磁盘上的侧链记录有智能体产生的每条消息。新的异步实例从该记录重放并从同步实例停止的地方继续。

自动后台化

当 `CLAUDE_AUTO_BACKGROUND_TASKS` 环境变量或 `tengu_auto_background_agents` GrowthBook标志启用时，前景智能体在120秒后自动后台化：

当通过环境变量或功能标志启用时，前景智能体在120秒后自动后台化。禁用时，函数返回0（无自动后台化）。

这是一个具有成本影响的UX决策。前景智能体阻塞父智能体终端——用户不能输入，不能发布新指令，不能生成其他智能体。两分钟足够智能体完成大多数快速任务同步（流式输出是有用的反馈），但足够短以至于长时间运行的任务不会劫持终端。

在fork实验下，自动后台化问题变得无关紧要：所有fork生成从一开始就强制异步。 `run_in_background` 参数完全从模式中隐藏。每个fork子智能体在后台运行，通过 `<task-notification>` 完成时报告，父智能体从不阻塞。

何时不使用Fork

Fork是几种编排模式之一，在三种情况下被故意排除：

协调器模式。 协调器模式和fork模式互斥。协调器有结构化的委托模型：它维护计划，向具有明确提示的工人分配任务，并跟踪进度。Fork的"继承一切"方法会破坏这一点。Fork的协调器会继承父协调器的系统提示（说"你是协调器，委托工作"），子智能体会尝试编排而非执行。 `isForkSubagentEnabled()` 函数首先检查 `isCoordinatorMode()` ，如果激活则返回false。

非交互式会话。 SDK和API消费者（`--print`模式、Claude Agent SDK）在没有终端的情况下运行。Fork的`permissionMode: 'bubble'`将权限提示显示到父智能体终端——在非交互式模式下不存在。不是构建单独的权限流，fork路径只是被禁用。SDK消费者使用显式的`subagent_type`选择代替。

显式的subagent_type。 当模型指定`subagent_type`（例如`"Explore"`、`"Plan"`、`"general-purpose"`）时，fork路径不被触发。Fork仅在`subagent_type`被省略时触发。这让模型可以选择"我想要具有自己系统提示和工具集的专业智能体"（显式类型）和"我想要一个上下文继承的克隆来处理并行"（省略类型）。

经济学

考虑一个具体场景。开发者要求Claude Code重构一个模块。父智能体分析代码库，形成计划，并行分派五个fork子智能体：一个更新数据库模式，一个重写服务层，一个更新路由器，一个修复测试，一个更新类型。

此时在对话中，共享上下文是实质性的：

- 系统提示：~4,000个token
- 工具定义（40+工具）：~12,000个token
- 对话历史（分析+计划）：~30,000个token
- 具有五个tool_use块的助手消息：~2,000个token
- 占位符工具结果：~500个token

总共享前缀：~48,500个token。每个子智能体指令：~200个token。

没有fork（五个独立智能体，每个都有新鲜上下文和自己的系统提示）：

- 每个子智能体处理自己的系统提示+工具+任务提示
- 无缓存共享（不同的系统提示、不同的工具集）
- 成本：5 x 完整输入处理

有fork（字节相同前缀）：

- 子智能体1：48,700个token全价（第一个请求缓存未命中）
- 子智能体2-5：48,500个token 10%价格（缓存命中）+ 每个200个token全价
- 子智能体2-5的有效成本：~4,850 + 200 = ~5,050个token等效每个

节省随上下文大小和子智能体数量扩展。对于具有100K token历史生成8个并行fork的热会话，缓存节省可能超过无共享时输入token成本的90%。

这就是为什么fork系统中的每个设计决策——线程传递而非重新计算、精确工具传递、占位符结果，甚至尽管被禁止仍在子智能体池中保留Agent工具——都针对一件事优化：字节相同前缀。每个决策都用少量优雅或安全性换取API成本的可测量降低。

设计张力

Fork系统做出了值得理解的明确权衡：

隔离与缓存效率。 Fork子智能体继承一切，包括可能与其任务无关的对话历史。重写测试的子智能体不需要父智能体讨论数据库模式设计的15条消息。但包含这些消息才是使前缀相同的原因。剥离无关历史会以破坏缓存为代价节省上下文窗口空间。设计打赌是缓存节省超过上下文开销。

安全性与缓存效率。 Agent工具保留在fork子智能体的工具池中，即使子智能体不得使用它。移除它会更安全（子智能体甚至不能尝试fork），但会改变工具数组序列化。样板标签和递归fork守卫是补偿控制——运行时预防而非静态移除。

简单性与缓存效率。 占位符工具结果是谎言。子智能体看到`'Fork started -- processing in background'`，无论父智能体的助手消息中的那些工具调用实际做了什么。这没关系，因为子智能体的指令告诉它要做什么——它不需要父智能体调度回合的准确工具结果。但意味着子智能体的对话历史在技术上是语无伦次的。占位符因其简洁和统一而被选择，而非准确性。

这些权衡中的每一个都反映相同的优先级：当你按token为大规模API调用付费时，字节相同前缀值得围绕它扭曲架构。

应用：为提示缓存效率设计

Fork智能体模式超越Claude Code推广。任何从相同上下文分派多个并行LLM调用的系统都可以从缓存感知请求构造中受益。原则：

1. 线程传递渲染的提示，不要重新计算。 如果你的系统提示包含任何动态内容——功能标志、时间戳、用户偏好、A/B测试变体——捕获渲染结果并通过值传递给子智能体。重新计算有分歧风险。

2. 冻结工具数组。 如果你的子智能体需要不同的工具集，你就在放弃工具块上的缓存共享。考虑保留完整工具集并使用运行时守卫（如fork样板的"不要使用Agent"）而非编译时移除。

3. 最大化共享前缀，最小化每个子智能体的后缀。 构造你的消息数组，使所有共享内容排在前面，每个子智能体的内容附加在末尾。交错共享和每个子智能体的内容会碎片化缓存边界。

4. 对可变内容使用常量占位符。 当消息结构需要响应以前的工具调用时，在所有子智能体中使用相同的占位符字符串而非实际（发散的）结果。

5. 测量盈亏平衡。 缓存共享有开销：每个子智能体更大的上下文窗口（它们携带无关历史）、运行时守卫而非静态安全、架构复杂性。计算你的并行模式（多少子智能体、共享前缀多大）在考虑额外上下文token后是否真的省钱。

Fork智能体系统，其核心是提示缓存利用引擎。它回答每个多智能体系统构建者最终面临的问题：当缓存给重复前缀90%折扣时，你会将架构重构多远来索取该折扣？Claude Code的答案是：非常远。